技術レポート

koshi - ターミナルネイティブ分散型SNS 技術レポート

バージョン3.1.3・サーバーレスP2Pメッシュ・ed25519認証・MITライセンス

game_ryo (ryopc)
バージョン 3.1.3・2026.10
MITライセンス

目次

01	エグゼクティブサマリー	3
02	背景と問題定義	4
03	アーキテクチャ	5
04	主要機能	7
05	CLIとWeb UI	9
06	APIリファレンス	10
07	デプロイと運用	11
08	セキュリティ	12
09	結論と今後の展望	13
10	付録	14

01 · EXECUTIVE SUMMARY

エグゼクティブサマリー

koshiは、データベースやサーバー構築を不要にし、**ed25519暗号による分散認証**と**P2Pゴシップネットワーク**を備えたターミナルネイティブの分散型SNSです。デフォルトではサーバーレスモードで動作し、SQLiteとhypercore/hyperswarmによるP2Pメッシュでオフラインファーストな投稿・DM・プロフィール同期を実現します。

核心 Takeaways

- サーバーレスP2Pメッシュがデフォルト。PostgreSQL不要で、インストール後すぐに利用可能。
- ed25519による分散認証とx25519エンドツーエンド暗号化DMで、プライバシーと改ざん防止を両立。
- Nostrプロトコルとの統合により、既存のNostrリレーへの投稿転送とイベント取得が可能。
- CLI(kbコマンド)とReact/Vite製Web UIの両方を提供し、ターミナルとブラウザの両方で操作可能。
- DockerスタンドアロンイメージおよびRender/Neonデプロイメントに対応し、運用コストを最小化。

本文档で答える3つの核心問題

分散型SNSを、サーバー構築不要・データベース不要・コマンド一発で始められるか?

ユーザーは自身のデータとアイデンティティを完全にコントロールできるか?

- 既存のNostrエコシステムと相互運用できるか?

02 · BACKGROUND

背景と問題定義

既存の集中型SNSはプラットフォーム事業者によるデータ管理・検閲・アカウント停止のリスクを伴い、ユーザーは自身のデータとアイデンティティを完全にコントロールできない。分散型SNSの代替案も多いが、サーバー構築やデータベース設定など技術的ハードルが高く、一般ユーザーが気軽に利用することは困難である。

当現状

現代のSNSは、少数の巨大プラットフォームによって集中管理されています。ユーザーの投稿、プロフィール、ダイレクトメッセージはすべて事業者のサーバーに保存され、利用規約の変更やアカウント停止によってデータへのアクセスを失うリスクが常に存在します。

核心問題

分散型SNS（Mastodon、Nostrなど）はこの問題に対する一つの回答ですが、多くのプロジェクトではサーバーの構築・運用、データベース（PostgreSQLなど）の設定、ドメイン取得、SSL証明書の管理など、技術的な専門知識が必要です。これにより、分散型SNSの恩恵を受けられる層が限定されています。

> koshiは、この「分散型SNSの利用ハードル」を下げることを目的としています。サーバーレスモードをデフォルトとし、SQLiteベースのローカルデータベースとP2Pメッシュネットワークにより、データベースや外部サーバーを一切不要にして利用を開始できます。

解決アプローチ

また、ターミナルネイティブなCLI（kbコマンド）を提供することで、開発者や技術者がすぐに分散型SNSを試用・運用できる体験を実現しています。npm install後にnpm linkするだけで、すぐにkb registerからアカウント作成と投稿が可能です。

03 · ARCHITECTURE

アーキテクチャ

koshiのアーキテクチャは、サーバーレスモード（SQLite＋P2Pメッシュ）とサーバーモード（PostgreSQL）の二つの運用モードを持ち、ユーザーのニーズに応じて選択できます。P2P層はhypercore・corestore・hyperswarmを使用し、認証層はed25519デジタル署名とJWTトークンを組み合わせています。

運用モード

koshiは、大きく分けて「サーバーレスモード」と「サーバーモード」の二つの運用モードをサポートします。サーバーレスモードがデフォルトであり、データベースやサーバー構築を必要とせず、ローカルのSQLiteデータベース（~/.config/koshi/local.db）にすべてのデータを保存します。サーバーモードではPostgreSQLをバックエンドデータベースとして使用し、マルチユーザーの共有インスタンスとして運用できます。

P2Pメッシュネットワーク

P2Pメッシュネットワークは、hypercore（追記専用ログ）、corestore（複数のhypercoreインスタンス管理）、hyperswarm（P2Pネットワーク発見と接続管理）のスタックで構成されています。投稿、DM、プロフィールはP2Pコアに追記され、接続されたピア間でゴシッププロトコルにより同期されます。

データフローは、1) ローカルAPIがSQLiteに書き込みと同時にP2Pコアに追記、2) P2Pブリッジが接続ピアへコアストア経由でレプリケート、3) 新規ピアがトピックに参加してアナウンスブロックを受信、4) ブリッジがP2PデータとローカルSQLiteを調整、という四段階で構成されています。

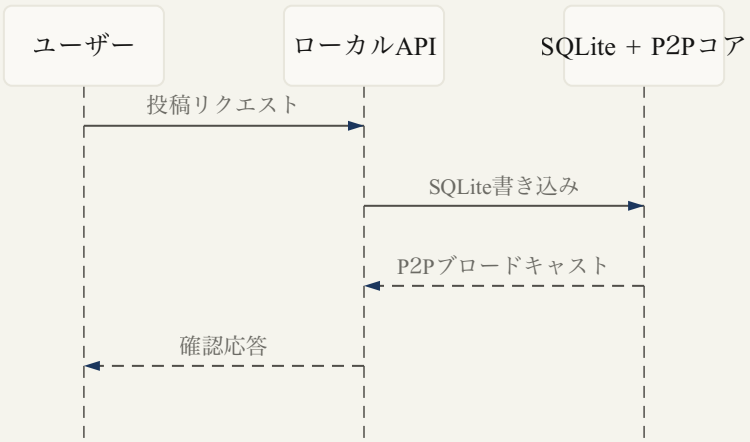


図 1: koshiのサーバーレスP2Pメッシュデータフロー

認証と暗号化

認証はed25519キーペアベースで、Nostrに触発された分散型認証モデルを採用しています。ユーザーは自身の秘密鍵を保持し、投稿やDMにはデジタル署名を付与することで改ざん防止と正当性証明を実現します。セッション管理にはJWT（HMAC-SHA256署名、24時間有効期限）を使用します。

DMはエンドツーエンド暗号化され、ed25519鍵からx25519鍵へ変換（ed2curve）した上でnacl.boxによる暗号化が行われます。送信者と受信者のみが復号でき、サーバーや中継ピアは内容を読むことができません。

04 · FEATURES

主要機能

koshiは、投稿・フォロー・エンドツーエンド暗号化DM・プロフィール編集・Nostr連携・P2P同期・マルチアカウント管理など、SNSとして必要な機能を一通り備えています。CLIとWeb UIの両インターフェースから利用可能です。

投稿とフィード

投稿機能により、ユーザーはkoshiボードにメッセージを投稿できます。投稿はed25519で署名され、P2Pメッシュ経由で接続ピアに拡散されます。フィード表示では、フォロー中のユーザーの投稿を優先的に表示するパーソナライズドフィードと、全ユーザーの投稿を表示するグローバルフィードを切り替え可能です。

フォローシステムとプロフィール

フォローシステムにより、特定ユーザーをフォロー・アンフォローでき、フォロー関係に基づいたフィードカスタマイズが行えます。プロフィール編集では、表示名・自己紹介・アバターを更新でき、自己表現の自由を確保します。

エンドツーエンド暗号化DM

エンドツーエンド暗号化DMは、x25519鍵による暗号化でプライベートメッセージの機密性を担保します。WebSocketベースのリアルタイムチャットUIにより、LINE風のバブルチャットをターミナルおよびブラウザで利用できます。

Nostrプロトコル連携

Nostrプロトコル連携により、Nostrキーペア(nsec/npub)の生成・管理、Nostrリレーへの投稿転送(push)、Nostrネットワークからのイベント取得(pull)が可能です。既存のNostrエコシステムとの相互運用性を確保しています。

P2P機能

P2P機能により、kb p2p status/start/stop/sync/posts/dmsコマンドでP2Pノードの状態確認、ピア一覧表示、投稿・DMの同期、手動同期トリガーなどが行えます。ブラウザからはP2Pに直接アクセスできませんが、kb serve＋Web UI経由でREST APIとWebSocketを介してP2P機能を利用できます。

マルチアカウントサポート

マルチアカウントサポートにより、複数のkoshiアイデンティティを単一CLIから管理でき、kb switchコマンドでアカウントを切り替えられます。各アカウントの認証情報は~/.config/koshi/config.jsonにJSON形式で保存され、ファイル権限は0600（所有者のみ読み書き）に設定されます。

05 · CLI AND WEB UI

CLIとWeb UI

koshiは、ターミナルネイティブなCLI(kbコマンド)と、React/Viteベースのモダンな Web UIを提供します。CLIは軽量でスクリプト化しやすく、Web UIは視覚的な操作とリアルタイム更新に適しています。

CLI(kbコマンド)

CLI(kbコマンド)は、bin/cli.jsをエントリーポイントとし、chalkによるターミナル色付けとoraによるスピナー表示で美しいCLI体験を提供します。登録・ログイン・投稿・フィード表示・フォロー・DM・チャット・プロフィール編集・アカウント削除・サーバー起動・リモート接続・Nostr操作・P2P操作・管理者コマンドなど、約20のサブコマンドを備えています。

主なCLIコマンドには、kb register(アカウント作成)、kb post(投稿)、kb feed(フィード表示)、kb follow/unfollow(フォロー管理)、kb dm/dms(ダイレクトメッセージ)、kb chat(リアルタイムチャット)、kb serve(ローカルサーバーレスノード起動)、kb connect(リモートサーバー接続)、kb nostr(Nostr連携)、kb p2p(P2P管理)、kb admin(管理者操作)などがあります。

Web UI

Web UIは、React 18とVite、React Router DOMを使用したシングルページアプリケーションです。フィードページ、プロフィールページ、DMページ、検索ページ、ログイン/登録ページ、Nostrページなどを備え、WebSocketによるリアルタイム更新に対応しています。ビルド済み静的ファイルはweb/ディレクトリに配置され、サーバーから自動配信されます。

クライアント側では、APIベースURLの動的解決により、サーバーレスモードとリモートサーバーモードを透過的に切り替えられます。/api/configエンドポイントからサーバーURLを取得し、接続先に応じたAPI呼び出しを行います。

06 · API REFERENCE

APIリファレンス

koshiはREST APIとWebSocket APIを提供し、クライアントからの操作を統一的に処理します。認証にはJWT Bearerトークンを使用し、レート制限や入力バリデーションによりセキュリティを確保しています。

認証エンドポイント

認証エンドポイント（POST /api/auth/register、POST /api/auth/login）は認証不要で、ed25519公開鍵による登録と署名チャレンジによるログインをサポートします。発行されたJWTトークンは24時間の有効期限を持ち、以降の認証が必要なエンドポイントでAuthorization: Bearer ヘッダとして送信します。

ユーザー・投稿・DMエンドポイント

ユーザーエンドポイントでは、プロフィール取得・更新・フォロー/アンフォロー・フォロワー/フォロー中一覧・ユーザー検索が可能です。投稿エンドポイントでは、フィード取得・投稿作成・単一投稿取得をサポートします。DMエンドポイントでは、受信箱取得・DM送信・既読マーク・未読数カウントが可能です。

管理者・P2P・WebSocketエンドポイント

管理者エンドポイントでは、ユーザー一覧・ユーザー詳細・ユーザー削除・管理者権限付与/剥奪が行えます（最初の登録ユーザーがローカルモードで管理者となります）。P2Pエンドポイントでは、P2Pノード状態・ピア一覧・P2P経由投稿・手動同期・P2Pノード起動/停止をWeb経由で制御できます。WebSocket接続（ws://host:port/ws?token={jwt}）により、post_created、dm_received、user_online/offlineなどのリアルタイムイベントを受信できます。

ヘルスチェックエンドポイント（GET /api/health）により、サーバーの稼働状況を監視できます。スタンドアロンサーバーでは、起動時に自動マイグレーションが実行され、必要なテーブルが作成されます。

07 · DEPLOYMENT

デプロイと運用

koshiはDockerによるスタンドアロン(SQLite)およびPostgreSQL連携のフルサーバー構成、Render.com＋Neon.techによるマネージドデプロイメント、および単純なnpm run serverによるセルフホステッドスタンドアロン運用に対応しています。

Dockerスタンドアロン構成

Dockerスタンドアロン構成は、PostgreSQL不要でSQLiteのみで動作する最も簡単なデプロイメント方式です。ghcr.io/ryopc/koshi:standalone-v3.1.2イメージをプルして実行するか、Dockerfileのstandaloneターゲットでビルドして実行します。~/.config/koshiディレクトリをボリュームマウントすることで、データの永続化を行います。

Docker PostgreSQL連携構成

Docker PostgreSQL連携構成は、runnerターゲットのイメージを使用し、外部PostgreSQLデータベース(DATABASE_URL環境変数で指定)とJWT_SECRET、ADMIN_USERNAME環境変数を設定して実行します。マルチユーザーの共有インスタンスに適しています。

Docker Compose構成

Docker Compose構成では、PostgreSQLコンテナとAPIコンテナを定義し、依存関係とボリューム永続化を管理します。開発環境および本番環境の両方で再現性のあるデプロイメントを実現します。

Render.com＋Neon.techデプロイメント

Render.com＋Neon.techデプロイメントは、NeonでPostgreSQLプロジェクトを作成し、RenderでGitHubリポジトリを接続して環境変数(DATABASE_URL、JWT_SECRET)を設定することで、mainブランチへのプッシュごとに自動デプロイされます。

セルフホステッドスタンドアロン運用

セルフホステッドスタンドアロン運用は、npm run server(またはnode src/local/server-standalone.js)で即座に開始でき、http://localhost:3000でREST API＋WebSocket＋Web UI＋P2Pメッシュが利用可能になります。個人ノードとして外部サービスに依存せず運用できます。

08 · SECURITY

セキュリティ

koshiは、ed25519署名、エンドツーエンドDM暗号化、JWT認証、レート制限、パラメータ化クエリ、Helmetセキュリティヘッダ、ハードコードシークレットの排除、CORS設定、ファイル権限制限など、多層的なセキュリティ対策を実装しています。

ed25519署名と改ざん防止

ed25519署名により、すべての投稿とDMは改ざん防止と正当性証明を備えます。ユーザーは自身の秘密鍵を保持し、公開鍵によるアイデンティティ検証が行われます。

エンドツーエンドDM暗号化

エンドツーエンドDM暗号化は、ed25519鍵からed2curve経由でx25519鍵に変換し、nacl.boxで暗号化します。送信者と受信者のみが復号でき、サーバーや中継ピアはメッセージ内容を読むことができません。

JWT認証とレート制限

JWT認証はHMAC-SHA256で署名され、24時間の有効期限を持ちます。認証エンドポイントはIPあたり10リクエスト/分のレート制限を設け、ブルートフォース攻撃を防ぎます。

SQLインジェクション防止と入力バリデーション

SQLインジェクション防止のため、すべてのデータベースクエリはパラメータ化クエリを使用します。入力バリデーションにより、ユーザー名フォーマット、コンテンツ長、鍵フォーマットなどを検証します。

Helmetによりセキュリティヘッダが有効化され、CORSはデプロイメントごとに設定可能です。ハードコードされたシークレットは排除され、すべて環境変数または設定ファイル(権限0600)で管理されます。

09 · CONCLUSION

結論と今後の展望

koshiは、サーバーレスP2Pメッシュによる分散型SNSの利用ハードルを大幅に下げ、ed25519認証とエンドツーエンド暗号化DMでプライバシーとセキュリティを確保する実用的な分散型SNSです。今後は、P2P機能のブラウザ対応強化、モバイルクライアント開発、より高度なモデレーション機能の追加などが検討されます。

v3.1.3の特徴

koshi v3.1.3は、サーバーレスP2Pメッシュをデフォルトとすることで、データベースやサーバー構築を不要にし、インストール後すぐに分散型SNSを利用開始できる点が最大の特徴です。SQLiteベースのローカルデータベースとhypercore/hyperswarmによるP2Pゴシップネットワークにより、オフラインファーストな投稿・DM・プロフィール同期を実現しています。

プライバシーとユーザー主権

ed25519による分散認証とx25519エンドツーエンド暗号化DMにより、ユーザーは自身のデータとアイデンティティを完全にコントロールできます。Nostrプロトコルとの統合により、既存のNostrエコシステムとの相互運用性も確保されています。

両インターフェースとデプロイメント

CLIとWeb UIの両インターフェースにより、開発者から一般ユーザーまで幅広い層が利用できます。Dockerスタンドアロンイメージおよび Render/Neonデプロイメントにより、運用コストを最小化しながら個人ノードや共有インスタンスを運用できます。

今後の展望

今後の展望としては、P2P機能のブラウザ直接対応(WebTransportやWebRTCの検討)、モバイルクライアント開発、高度なモデレーション機能(スパムフィルタ、通報システム)、マルチデバイス同期の強化などが考えられます。

今すぐ始める

分散型SNSを、サーバー構築不要・データベース不要・コマンド一発で始められる。それがkoshiの提供する価値です。

10 · APPENDIX

付録

A. 参考資料

- koshi README.md (v3.0.0, https://github.com/ryopc/koshi)
- koshi memo.txt (作業メモ, 2026-10-14更新)
- koshi package.json (v3.1.3)
- koshi Dockerfile (standalone/runnerマルチステージビルド)
- noble/ed25519 - Pure JS ed25519 implementation
- tweetnacl - Cryptographic primitives
- ed2curve - ed25519 to x25519 key conversion
- hypercore - Append-only logs
- corestore - Hypercore factory
- hyperswarm - P2P network discovery
- nostr-tools - Nostr protocol library
- better-sqlite3 - Fast SQLite bindings
- Express - Web framework
- React 18 + Vite - Web frontend

B. 略語集

CLI: Command Line Interface(コマンドラインインターフェース)

DM: Direct Message(ダイレクトメッセージ)

JWT: JSON Web Token

P2P: Peer-to-Peer(ピアツーピア)

REST: Representational State Transfer

SQLite: 軽量リレーショナルデータベース

WebSocket: 双方向通信プロトコル